Filter provenance, deployment, and response strategy

v0.93.0-18-g1106ca6-dirty (2026-08-28)

This document defines how a room-correction filter becomes a deployable, auditable bundle, how that bundle reaches the playback machine, and how `omdrcctrl` displays the measured, filter, and corrected responses belonging to the coefficients BruteFIR is actually using.

The control panel's `/configuration` page is the normal live-install path and invokes this same audit/build engine itself. The command-line and Git handoff documented below is an alternative for offline publication or machine provisioning, not a procedure to run after a web install. The web path archives every input in a persistent design root, requires a deployment commit when that root is already a Git repository, supports a plain folder otherwise, and only then derives the installed runtime from that authoritative bundle.

There are two central rules:

> A graph is selected by the hashes of the coefficient files loaded by the running BruteFIR process, never only by a geometry name, sample rate, filename, or UI selection.

> Every plotted curve is one REW text export, drawn exactly as REW wrote it. Nothing between REW and the browser averages, sums, convolves, interpolates or smooths a response.

The second rule is what makes the first one useful. A calculated prediction can be correct and still disagree with what the designer sees in REW, and then the two tools cannot be compared at all. Eight exports go in, eight curves come out; the operator reads the same numbers in both places.

Hashes prove byte identity and protect an established relationship. They cannot retroactively prove the human design decision that a filter was made from a particular measurement, and they cannot bring the measurement back. That relationship is asserted by the names the designer gives the exports in one directory and frozen by hashing them; where those exports came from is recorded separately, as the hash of the REW `.mdat` behind them and the commit of the project that holds it.

1 Audit of `120.blue` on 2026-08-13 (historical)

> This section records an audit performed under the schema 1 tooling, which declared roles on a command line and anchored a bundle to a Git tag. It is kept as evidence for how that base bundle came to exist. The procedure it describes has been replaced by the single dir-driven command documented under *Deployment command and transaction*.

The adjacent source repository was `../DRC/DRC-120.blue` at commit `d7a8e4dbf9418e43e2397bb140a8f81edcab1087`. The selected base source files are tracked and unmodified. The source repository also contains unrelated untracked files, so its complete worktree is not clean.

Update on 2026-08-14: the adjacent checkout then contained a newer, uncommitted candidate — the FLX/FRX trimmed WAVs and many files under `new.filters.txts` were modified, and no annotated tag named that candidate.

1.1 Base filter: verified deployment lineage

The source WAVs copied into the site repository are byte-identical:

Channel	Source and repository copy	SHA-256
Left	`FLX-trimmed-48k.wav`	`4116b567012c0fb857b9a7f1a5a70e5dca16188aa6ea6b189289c174f3ba7170`
Right	`FRX-trimmed-48k.wav`	`40a28cc294e94532b17edd5ab24a867c9a7cefb74af301e115fd6227fde547c6`

Both are mono, 48,000 Hz, signed 32-bit PCM and contain 131,072 samples. `filters/120.blue/48000/L.raw` and `R.raw` are exact, sample-for-sample float64 conversions of those WAV samples.

All five committed rate pairs were also regenerated in a temporary directory with the current `scripts/REW2raw-all-rates.sh`. Every regenerated RAW file was byte-identical to the committed file:

Rate	Left SHA-256	Right SHA-256
44,100	`afcc63a4d856008b9c68739de60b14b6967e289a65b844c9e61ca687f4952db9`	`d02dfed9f20fd4a07c5b0d5dfa9dead5ac7d9a8d4d660a9496413d93d5630459`
48,000	`e0881d86077f25413c1c010968b0fa2840e5a1f169b91a131d60de18e79085d1`	`31a94f64585cdd398c2d3d3042b360bc2eeb7f8bfed2a6191b8f189752acfd30`
88,200	`f5d7775909f6023708ba3c9ae00b8644c4d9db7d8e7f6969030d80c8866f4cb0`	`68cf38e37c46e43e6da955edaf934b439e04093caf8b916cfc143ddc756c7df7`
96,000	`986a7d4f5a72402c89a29ba1e32eedb3f480030b52971e0ac73c026265b68abd`	`5e076e36f1fe8b3b90ec3f806740c0f21e8fa50011fe62b052b5e2846c6d49f0`
192,000	`d5eda1b8aca4247e74886424d1bc18b65cda0f8467d3ed0a7d2b28d1e9920bf7`	`79d63e8eb46fd533314a608e24497d0cb16ac3005b79f86fb32590e44cefb4df`

The five base BruteFIR templates select the matching rate directory, `L.raw` and `R.raw`, and declare `FLOAT64_LE`. Therefore the chain from the committed base 48 kHz WAVs to every deployed base RAW is reproducible and verified.

1.2 Filter TXT-to-WAV correspondence

The filter response exports have these hashes:

File	SHA-256
`txt/FLX-trimmed.txt`	`69bfd876173f7bb5c39876d547da02d4132d819eb27aacc25cc6985ea95f640e`
`txt/FRX-trimmed.txt`	`da255d770d0c2b39257d3f14a6f1c9d8ecd4dd11d87b76159e50d8a0a174d2a8`

Each TXT has 65,533 response rows. Its frequency values map exactly, within the printed decimal precision, to FFT bins 3 through 65,535 of its corresponding 131,072-sample WAV. The WAV has a 24,000-sample (500 ms) causal export delay. After removing exactly that delay, the WAV FFT agrees closely with the exported magnitude and phase. Across the complete export the RMS errors are 0.0125 dB and 0.0946 degrees for FLX, and 0.0018 dB and 0.0119 degrees for FRX. Above 100 Hz, the worst errors are 0.049 dB/0.454 degrees for FLX and 0.040 dB/0.400 degrees for FRX.

The largest FLX differences are around 27–31 Hz (up to 1.116 dB and 13.28 degrees). They are consistent with exporting a finite, windowed causal impulse from a frequency-domain response; consequently the TXT and WAV are not expected to be two byte-equivalent encodings of identical numbers. A future export should also include a response TXT made by re-importing the final WAV into REW. That second TXT provides an independent, post-window reference that can be compared much more tightly to the Python FFT.

1.3 Historical base-bundle REW audit (not a deployment dependency)

The earlier audit of the already deployed default bundle used `120-blue-with-inversion.mdat`, SHA-256 `b184b824236868a898c33877a56d0f1003a2e442922d8b4f9c05ef1a51b8d6c7`. It was opened with REW V5.40 Beta 132 using `-nogui -noaudio -api`; no audio device was opened. All TXT exports resolve to unique traces in its 44-trace inventory.

The original `L.txt` and `R.txt` hypothesis is wrong: those are older January 2025 measurements. The actual source pair and independent sum were measured with the same setup and acoustic reference within 109 seconds:

Role	Trace / UUID	TXT SHA-256
Left	`L.120.Blue` / `51ba95b5-1b8e-4303-8c8d-f1828cd75463`	`aa829ea8d0ef4ae97802f9c88fe7e53688c0b694f3c1bcf09816ce57331804f1`
Right	`R.120.Blue` / `1f7ef644-083c-47c2-b65f-e116b89f3d50`	`56e11fdb76bcc0f5d729fed3eb446fa6c2af3c29bee4c808d340b9fa928c8d58`
Independent sum	`L+R.120.Blue` / `d0bbd016-69e2-43a8-9c92-822618602af7`	`6555b6bfbad78c93b315b47861d8940b55feb47d38f5b48cce00a19a01301d56`

The project notes preserve the arithmetic chain: L/R are multiplied by `X801 (revised)`, converted to minimum phase, inverted against the RMS/phase-average target with 0 dB maximum gain over 20–225 Hz, converted to `LFilter`/`RFilter`, multiplied by `X801`, and trimmed to `FLX-trimmed`/`FRX-trimmed`. The final trace UUIDs are `c2515dc4-51d7-4e96-9d3a-2be205738808` and `7815c6db-1e74-4957-8573-f0fa419e2b32`.

REW's API impulse arrays for those final traces match the source WAVs within float32-percent to PCM-S32 quantisation: maximum absolute errors are `1.43e-8` left and `2.15e-8` right. This closes the semantic gap from measured traces, through the recorded REW operations, to the deployed source WAVs.

This paragraph is historical evidence for that base bundle. Neither deployment nor plotting opens an `.mdat`: schema 2 hashes the project file and records its Git blob as recoverability evidence, but the exported files in the design directory remain the numerical trust boundary.

1.4 What must not receive a green status

1. The base pair needs at most 2.3 dB attenuation (including the 1 dB safety margin); all base configurations specify 3.0 dB and pass. `scripts/headroom_calc.py filters/120.blue` discovers the real rate pairs.
2. Any selector without a manifest is **unverified**, whatever its audio quality. The web UI reports it as such and draws no curve at all — not even a diagnostic FFT of the live coefficients, because that would be a calculated curve on a page that promises exported ones. A RAW pair that cannot be reproduced byte-for-byte from its recorded source exports is the usual cause: it must be re-exported and redeployed as a new bundle before it can be trusted.

2 Provenance bundle

Each immutable design inside a geometry needs one manifest. The default design keeps the legacy paths; additional A/B designs use an `@design-id` selector:

```
filters/<geometry>/
  provenance/
    rscreen-20260812.json           # the bundle manifest; written last
    rscreen-20260812.source.json    # the recipe that reproduces it
  source/
    rscreen-20260812/               # the ten input files, under their own names
      L.txt  R.txt  LR.txt
      FLX-trimmed.txt  FRX-trimmed.txt
      L.filtered.txt  R.filtered.txt  LR.filtered.txt
      FLX-trimmed-48k.wav  FRX-trimmed-48k.wav
  analysis/
    rscreen-20260812.json
  44100/L.raw
  44100/R.raw
  44100/@rscreen-20260812/L.raw
  44100/@rscreen-20260812/R.raw
  ...
configs/<geometry>/
  brutefir-44100.conf.in
  brutefir-44100@rscreen-20260812.conf.in
```

Bundle source names mirror the input directory exactly, because those names are now the role assignment. Copying the selected sources into the site repository is intentional: a deployment must not depend on a mutable sibling directory or an absolute path that will not exist on the installed machine.

The manifest contains:

- schema version, geometry, design ID, description, audit date and a content-derived `bundle_id`;
- the directory the exports came from, and for every one of the ten inputs its role, name, bundle path, byte size, SHA-256 and Git blob ID;
- the project that produced them: repository path, remote, branch, HEAD commit, commit date and subject, the repository-relative path of the export directory, and whether every input was committed at deployment time;
- the REW `.mdat` those exports were taken from: name, path, byte size, SHA-256 and Git blob ID;
- the aggregate convention this design was built around, taken from the filenames rather than asserted on a command line;
- parsed REW header metadata per export: measurement name/date, source, notes, smoothing, frequency step and REW version;
- TXT/WAV validation results, headroom per channel, safety margin and required attenuation;
- the exact runtime rate/design mapping and the expected BruteFIR format and attenuation.

The `bundle_id` is the SHA-256 of a canonical identity containing a hash of the complete source object (directory and every artifact record), all source artifact hashes, runtime config/RAW hashes and settings, and the analysis hash. Editing any provenance shown in the UI therefore invalidates the bundle instead of changing a cosmetic label.

Schema 2 dropped the *tag* layer schema 1 carried: a design is no longer gated on an annotated tag object and a separately committed declaration file. What survived, and what schema 2 makes mandatory instead, is the pair of questions a hash cannot answer — *which project produced these numbers* and *which measurement session are they a view of*. Those are recorded as a commit ID and a `.mdat` hash, and both live inside the hashed `source` object, so neither can be edited after publication without invalidating the bundle.

3 The design directory

REW exports a measurement session into one directory beside the `.mdat` it came from, inside a project directory named for the geometry. That is the layout the command reads:

```
DRC-120.blue/                       the project: one geometry, under Git
  120.blue.Rscreen.mdat             the measurement session
  120.blue.Rscreen.txts/            everything REW exported from it
```

Inside the export directory, the file names carry every meaning that used to be a command line switch. A bare name is the room as it is; a suffix says what was done to it:

```
  L.txt                   measured left, before correction
  R.txt                   measured right, before correction
  LR.txt                  measured pair — REW's vector average
    or L+R.txt            measured pair — the sum
  FLX-trimmed.txt         exported response of the left filter
  FRX-trimmed.txt         exported response of the right filter
  FLX-trimmed-48k.wav     deployable left impulse
  FRX-trimmed-48k.wav     deployable right impulse
  L.filtered.txt          REW's filtered result for the left channel
  R.filtered.txt          REW's filtered result for the right channel
  LR.filtered.txt         the filtered pair, with LR.txt
    or L+R.filtered.txt   the filtered pair, with L+R.txt
    or L+R.remeasured.txt the room measured again with the DRC running
```

A trailing `.txt` is optional and matching ignores case, so REW's own `L.Filtered.txt` and a hand-typed `l.filtered` name the same thing. Two files that could fill one role is an error, not a silent preference.

Three claims are being distinguished, and the suffix is what distinguishes them:

name	what it is	who produced it
`L.txt`, `R.txt`, `LR.txt`, `L+R.txt`	the room before correction	a sweep
`*.filtered.txt`	the measurement with the filter applied to it	REW
`L+R.remeasured.txt`	the room after correction	a second sweep

A re-measurement is always L+R: both speakers played into the microphone, so what came back is a sum. There is no `LR.remeasured` — a vector average is something REW computes from two separate sweeps, not something a room can perform. The command refuses that name outright. `L+R.measured.txt` is still accepted for files that already carry it, with a yellow note recommending `.remeasured`, because "measured" on its own reads like an uncorrected sweep.

LR versus L+R is the whole aggregate convention. `LR.txt` says the design was built around REW's vector average; `L+R.txt` says it was built around the sum. Mixing them — `LR.txt` beside `L+R.filtered.txt`, or `LR.txt` beside `L+R.remeasured.txt` — is refused, because the two curves would then be measuring different things and a graph drawing one against the other would be a lie. Nothing recalculates the aggregate to match the name: the name only decides the legend wording.

3.1 What every export must be

Two properties are required of all eight text exports, and a design that lacks either is refused rather than annotated:

- **Unsmoothed.** `* Smoothing: None` in the header. REW's smoothing is baked into the numbers it writes; nothing downstream can undo it, so a smoothed export would make the page draw a smoothed curve while promising a measurement. An export that states no smoothing at all is refused too — it cannot be *shown* to be unsmoothed. The Smoothing selector in the browser is a separate thing entirely: it works on a copy, at view time, and is reversible.
- **From a measurement at 48 kHz or below**, which is checked as *the export must not reach past 24 kHz*. The deployed coefficients are all resampled from one 48 kHz impulse; a wider measurement would put a corrected curve on the page describing a band the filters were never designed against.

The command lists every offending file with the reason, at the second step, before anything is hashed or written.

A third check belongs with these and is described under the transaction below: each filter TXT must be provably the exported response of the impulse WAV that becomes the runtime coefficients. Together the three mean a green page cannot be showing a smoothed curve, an out-of-band measurement, or a filter response that does not describe the bytes BruteFIR loaded.

Geometry and design ID come from the path. `DRC-120.blue/120.blue.Rscreen.txts` reads as geometry `120.blue`, design `Rscreen`: the leading `DRC-` and the trailing `.txts` are stripped, and a geometry prefix on the session name is dropped. A nested `<geometry>/<design-id>/` pair and a single `<geometry>@<design-id>` directory are read the same way. `--geometry` and `--design` override whatever the path suggests — use them when the design ID should carry a date, as `rscreen-20260812` does. `default` is reserved.

4 Where a design comes from, and how to get it back

A content hash proves a bundle plots what it claims. It cannot say where those numbers came from, and it cannot bring them back once the working directory has moved on. Two records close that gap, and one commit keeps the result retrievable.

4.1 The measurement session

Every export in the directory is one view of a single REW file. That `.mdat` is located automatically — the sibling matching the directory name, or the only `.mdat` inside it, or `--mdat` — hashed, and recorded in the manifest with its Git blob ID. Given a deployed bundle, the exact sweeps that produced the filters come back with:

```
git -C ../DRC/DRC-120.blue cat-file blob <git_blob> > 120.blue.Rscreen.mdat
```

A design whose session cannot be identified is refused, and `verify_filter_bundle.py` rejects a manifest that does not name one.

4.2 The project

The export directory has to live in a Git work tree, and every one of the ten inputs and the `.mdat` has to be committed — staged is not enough, because only a commit makes the bytes fetchable later. The manifest then records the repository, its remote, the branch, the HEAD commit and subject, and the repository-relative path of the export directory. This is the link between `omdrc-801N/filters/120.blue/...` and the project `DRC-120.blue` that produced it, and it is the one thing in the chain that no hash could have supplied.

Refusing lists exactly which files are uncommitted and prints the `git add` and `git commit` lines to run. `--allow-uncommitted` proceeds anyway and writes `clean: false` plus the offending paths into the manifest, so the gap is recorded rather than hidden; the web UI shows it, and the verifier warns.

4.3 The room's history

The room directory — `../omdrc-801N` — is itself a Git repository, and that repository *is* the deployment history. After a successful deployment the command stages `filters/<geometry>` and `configs/<geometry>` and makes one commit:

```
Deploy 120.blue @rscreen-20260812 (bundle 5bbafbf6230f)

Geometry:     120.blue
Design:       rscreen-20260812
Bundle:       5bbafbf6230fb348819eb2b875ca042ebe8db4028b8d6a6736e7a40f6d03c5a4
Aggregate:    LR (filtered after correction)
Rates:        44100 48000 88200 96000 192000
Project:      DRC-120.blue @ 803426fd0137
Exports:      120.blue.Rscreen.txts
Measurements: 120.blue.Rscreen.mdat sha256 a19a39f24c5124c4
  restore:    git -C <project> cat-file blob 41fabea2… > 120.blue.Rscreen.mdat
```

`git log` in the room is then the list of every filter set that was ever live, and any of them comes back exactly as deployed:

```
git -C ../omdrc-801N log --oneline -- filters/120.blue
git -C ../omdrc-801N checkout <commit> -- filters/120.blue configs/120.blue
```

Redeploying identical bytes makes no empty commit. `--no-commit` skips the step, and a room that is not a work tree deploys with a warning that says what is lost and how to fix it (`git init`). Nothing about verification depends on this commit: it is how you travel back, not how a bundle is trusted.

5 Deployment command and transaction

One command, one directory:

```
python3 scripts/new_filter_design.py ../DRC/DRC-120.blue/120.blue.Rscreen.txts
```

It reports what it found, asks, and only then writes. `--dry-run` runs every check including the SoX conversions and stops without asking; `--yes` skips the prompt for scripted use, and refuses to assume consent when standard input is not a terminal. `--geometry`, `--design`, `--mdat`, `--rates`, `--safety-margin`, `--attenuation`, `--site-root`, `--replace-design`, `--allow-uncommitted` and `--no-commit` remain available with sensible defaults; no option names one of the ten inputs or asserts what one contains.

The options that weaken a guard are deliberately explicit:

Option	Effect	What it does not do
`--dry-run`	Performs discovery, parsing, TXT/WAV validation, SoX conversion, headroom, config bake/read-back, and manifest construction in private staging	Writes, asks, commits, installs, or selects nothing
`--yes`	Supplies confirmation for a reviewed non-interactive invocation	Does not bypass any content or provenance check
`--replace-design`	Permits an existing design ID's differing runtime/config bytes to be replaced	Does not skip verification or allow a partial L/R publication
`--allow-uncommitted`	Publishes even though source exports or the `.mdat` cannot be recovered from the recorded project commit	Records `clean: false`; the verifier and UI keep the warning visible
`--no-commit`	Leaves the published site-tree changes uncommitted	Does not change bundle verification; it gives up automatic deployment history
`--require-clean-site`	Requires `--site-root` to be its own clean Git work tree before publication	Does not commit or discard pre-existing work
`--require-commit`	Fails unless the publication is recorded in site history	Does not weaken source or bundle verification
`--site-root DIR`	Chooses the one site checkout to read and write	Does not change CMake's separate `OMDRC_SITE_DATA_DIRS` search path

The transaction is:

1. Resolve every role to exactly one regular file in the directory. Reject symlinks, missing names, ambiguous spellings and contradictory aggregate pairs, naming the offending files in colour.

2. Parse the eight exports and record each one's REW headers. Refuse any that carries REW smoothing, states none, or reaches past 24 kHz — the two properties above, checked on all eight at once so every offender is named in one message.
3. Check each filter TXT against its impulse WAV in complex frequency space after detecting one integer causal delay and one constant export gain. Store both transformations, thresholds and residual errors; reject frequency-dependent discrepancies. This is the only DSP in the pipeline, and it exists to prove the plotted FLX/FRX curve describes the bytes BruteFIR will load.
4. Find and hash the `.mdat`, then require the project to be a Git work tree with every input and the session committed. Record the commit, the remote and the blob IDs.
5. Print the curves, the project and session, the measurement headers, the filters and every path that will be written, then ask for confirmation.
6. Build everything in a new staging directory: copy canonical sources, generate every requested rate with SoX, calculate headroom, write graph data and render the candidate manifest.
7. Re-read and hash every staged artifact. Parse each candidate BruteFIR config and require its rate, format, two coefficient paths and attenuation to match the manifest, with configured attenuation at least the calculated requirement.
8. Publish the staged bundle only after all checks pass. Never update half of an L/R pair or one rate at a time. The manifest is written last as the commit marker, and only then is `source/<design>/` and each `<rate>/@<design>/` pruned down to exactly what that manifest names — a design owns those directories, so a redeployment under different filenames leaves nothing behind. A bare `<rate>/`, shared with the `default` set, is never touched.
9. Commit the result in the room repository, then run `scripts/verify_filter_bundle.py --all` in CI and before installation. The verifier checks hashes, manifests, config mappings, analysis dependencies, the named session and headroom without modifying files.

The terminal presentation is shared by `console_ui.py`: stages, coloured diagnostics, confirmation, and the final `FAIL:` shape are consistent between publication and removal, and colour disables itself for pipes or `NO_COLOR`. `filter_workflow_next.py` then prints the repository-to-runtime handoff with an explicit working directory for every command when the engine and site data are separate checkouts.

6 Deploying a verified design to the playback machine

Publication changes the site repository; it does not change an installed machine. The complete handoff crosses four independently checkable boundaries:

Boundary	Durable evidence	Check
REW project -> published bundle	Source-project commit, one blob per input, `.mdat` SHA-256	`new_filter_design.py` refuses uncommitted input unless explicitly overridden
Published bundle -> site history	One commit containing `configs/<geometry>` and `filters/<geometry>`	`git log -- configs/<geometry> filters/<geometry>`
Site checkout -> installed runtime tree	CMake-selected source directory and read-only bundle verification	Configure fails before install on a bad manifest, hash, config mapping, source copy, or headroom result
Installed tree -> audible identity	Running BruteFIR config plus exact L/R RAW hashes	The Filter response page goes green only for one exact manifest match

On the design machine, `new_filter_design.py` commits the site repository by default. Re-verify the committed bytes, then push that repository:

```
export OMDRC_SITE_ROOT=~/devel/omdrc-801N
python3 scripts/verify_filter_bundle.py --all --require-sources
git -C "$OMDRC_SITE_ROOT" status --short
git -C "$OMDRC_SITE_ROOT" push
```

On the playback machine, pull the site repository and ensure `host.cmake` names it in `OMDRC_SITE_DATA_DIRS`. A first configure must initialise a fresh build tree with `-C`; an existing correctly initialised tree can be reconfigured normally:

```
git -C ~/devel/omdrc-801N pull

# First build for this host:
cmake -C host.cmake -S . -B build

# Later deployments may reconfigure the existing cache:
cmake -S . -B build

cmake --build build
sudo cmake --install build
```

During configure, CMake prints which checkout supplied every geometry and runs `verify_filter_bundle.py --require-sources --no-next --site-root <winner>` over its manifests. Installation copies only the runtime RAWs, rendered configs, manifests, and analysis JSON; the source exports and `.source.json` recipe stay in the site repository.

Restart the control service so it reads the installed metadata, select the geometry and immutable design, then check the runtime identity:

```
# FreeBSD (use systemctl restart omdrcctrl on Linux)
sudo service omdrcctrl restart

/usr/local/bin/omdrc geometry 120.blue
/usr/local/bin/omdrc design --list
/usr/local/bin/omdrc design @rscreen-20260812
```

Finally open `http://<box>:9090`, enter **Filter response**, and require the green identity to show the selected geometry/design and the same complete `bundle_id` printed by the publication/verifier. A missing or different ID is not a cosmetic deployment lag: it means the displayed curves must not be trusted. The NEXT block printed by the scripts is the host-specific, copy/paste version of this sequence and should take precedence over the generic paths above.

7 Removing a design

A design is spread across a manifest, an analysis file, ten source copies, one coefficient pair per rate and one config template per rate. Deleting some of those produces something worse than a deployed design: still listed by `drc.sh design --list`, still offered by the web remote, and no longer verifiable. One command removes all of it:

```
python3 scripts/remove_filter_design.py --list
python3 scripts/remove_filter_design.py 120.blue@rscreen-20260812
```

The selector is the one the UI shows. It reports what the design was — bundle ID, project and commit, REW session, rates — then every path and byte count it will delete, and asks. `--dry-run` and `--yes` behave as they do when deploying.

Deletion order matters as much as publication order, and inverts it. **The manifest goes first.** While it exists it is the claim that every other file is present, so removing it is what makes the design cease to exist for every reader; nothing can then meet a manifest naming a file that is already gone. The un-suffixed `default` set is refused outright: a geometry falls back to it, so removing it would take the geometry with it, and the command says so instead of obeying.

The removal is then one commit in the room repository, listing every path it took away. The design stays recoverable from the deployment commit that introduced it — removing a design deletes files, not history:

```
git -C ../omdrc-801N log --oneline -- filters/120.blue
git -C ../omdrc-801N checkout <deploy-commit> -- filters/120.blue configs/120.blue
```

Nothing about this reaches an installed machine by itself. Re-run CMake, `make install` and restart the service; the command's **NEXT** section prints the exact sequence, warns when a geometry is left with no immutable design, and reminds you that a BruteFIR already running the removed coefficients keeps playing them until it is restarted on another design.

7.1 A note on the retired declaration step

Earlier versions split this into `declare_filter_design.py`, a Git commit, an annotated tag, and `new_filter_design.py`. Twelve `--measurement-left`-style switches assigned roles that a filename can state just as precisely and far more legibly, and the tag object added a second, manually maintained name for something the content hash already identifies. Both scripts, and `filter_design_suggest.py` with them, are gone. Git remained where it is irreplaceable: naming the project a design came from, and keeping every deployed filter set retrievable.

8 What the analysis file contains

There is no offline response calculation. The analysis file is a transcription of eight REW exports:

Trace	Group	Source file	Visible by default
Original L	Original	`L.txt`	no
Original R	Original	`R.txt`	no
Original aggregate	Original	`LR.txt` / `L+R.txt`	yes
Filter FLX	Filter	`FLX-trimmed.txt`	no
Filter FRX	Filter	`FRX-trimmed.txt`	no
Corrected L	Corrected	`L.filtered`	no
Corrected R	Corrected	`R.filtered`	no
Corrected aggregate	Corrected	`LR.filtered` / `L+R.filtered` / `L+R.measured`	yes

Each trace records the export it came from, its magnitude column rounded to the three decimals REW writes, and its phase column rounded to the four REW writes. The phase is not re-wrapped: a value REW wrote as `+180.0000` stays `+180.0000`.

Exports do not share one grid — REW's filter and corrected traces routinely have a different row count from the room measurements — so each trace names the grid it was written on. Grids that are bit-identical are stored once and referenced by several traces. That is an encoding choice only; no trace is ever resampled onto another trace's grid.

Two things that earlier versions calculated are deliberately absent:

- **the aggregate.** `original-sum-calculated`, the complex `M_L + M_R` curve, is gone. If you want to see an aggregate, export it from REW.
- **the prediction.** `corrected = M * F` by complex multiplication is gone, and so is the FFT-of-the-WAV filter curve that fed it. The corrected curves are REW's own `L.Filtered` / `R.Filtered` exports, and the filter curves are REW's own `FLX-trimmed` / `FRX-trimmed` exports.

BruteFIR's coefficient attenuation is **not** subtracted from the filter curves either, although it is part of the audible runtime transfer:

```
F_runtime = F_raw * 10 ** (-attenuation_dB / 20)
```

Applying it would move the plotted curve away from the exported one, which is exactly what this page exists not to do. The attenuation is stated in the note under the graph and in the details panel instead, so the reader can account for it without the data being altered.

9 Runtime and web UI binding

The existing `omdrcctrl` implementation already locates the running BruteFIR process, parses its exact `.conf`, reads the coefficient paths, and exposes a **Filter response** button. Extend that path rather than adding a second, geometry-selected source of truth.

For each request:

1. Read the `.conf` used by the running BruteFIR process.
2. Hash the exact L/R RAW files named by that config while reading them. Cache by path, inode, size, and high-resolution modification time only as a performance optimisation.
3. Find a manifest containing that exact `(relative path, SHA-256, rate, format)` pair. Verify the analysis-file hash and its recorded input hashes.
4. Check the config attenuation against the manifest.
5. Release the stored traces **unmodified** — no scaling, offset or resampling between the analysis file and the browser — with a verification object and bundle details. Never label a response verified merely because `geometry` and `rate` match.
6. Put the provenance constraint in the always-visible green banner. The expandable details show the bundle ID, active L/R RAW hashes, the directory the exports came from, the project and commit that produced them, the REW session behind them, and each export's name and hash.

Use two visible runtime states:

- **Verified** (green): active RAW hashes, config, analysis and source binding all pass, and the stored exports are drawn;
- **Mismatch / unverified** (red): no manifest matches the active bytes, a hash differs, the config differs, or analysis dependencies fail. **No graph is drawn.** The page previously fell back to a live FFT of the active RAW; that fallback is gone, because a calculated curve on a page whose entire promise is "these are REW's numbers" is worse than no curve.

The control page applies the same rule to A/B switching. During a switch it shows the previous and requested selectors. After `drc.sh` returns, the server re-reads the BruteFIR process, parses the config actually in use, hashes its L/R RAWs and reports the new selector as verified only if that runtime identity matches one manifest. A new immutable `@design-id` that starts but fails this check is an assurance failure, not a successful green switch. Legacy variants may still run, but remain visibly unverified.

Session persistence has one source of truth: `drc.sh`'s `last_geometry`, `last_arg` (mode/rate plus design selector), and `last_power`. Successful switches update those files automatically; the browser does not maintain a second copy. `drc.sh session` exposes their effective restore tuple as stable `key=value` output. The control page shows both the exact active identity and the auto-saved tuple, enables **Restore saved** only when they differ, and post-verifies the active config after restoration before claiming success. Restore deliberately honours a saved `off` state.

The page uses one Chart.js canvas with a **Magnitude / Phase** toggle and a custom checkbox legend shared by both modes. Default to only the two aggregates — original and corrected. The six per-channel and filter traces stay one tap away.

A **Smoothing** selector offers unsmoothed, variable, psychoacoustic, 1/3 and 1/6 octave. It defaults to **unsmoothed**, so what loads is what REW exported. Smoothing is computed in the browser on a copy of the arrays, for viewing only: the stored data, its hashes and the served JSON are untouched, and switching back to unsmoothed restores the exported numbers exactly. This is the one place in the system where a displayed curve may differ from an exported one, it is always named in the chart subtitle and the note under the graph, and it is never the default.

Smoothing REW itself applied before export never reaches this page: a smoothed export cannot be deployed at all. The header value is still recorded per measurement in the analysis file and shown in the details panel, where it reads `None` for every trace — evidence of the check rather than a caveat about the data.

A details panel shows the verification badge, bundle ID, active config/rate/attenuation, short L/R hashes, the source directory, the source project and its commit, the `.mdat` the exports were taken from with its hash, the aggregate convention and whether the corrected aggregate was filtered or re-measured, every plotted export with its hash, exported measurement headers/notes and REW smoothing, the TXT/WAV residuals and headroom. A bundle published with `--allow-uncommitted` says so here and on the control panel line, because a design whose exports were never committed cannot be reproduced.

10 Where the site data lives

`configs/<geometry>` and `filters/<geometry>` are site data: one physical room's measurements, of no use to anyone else. They are resolved through a *site root* rather than assumed to sit in the engine checkout, so a room can be its own repository while the engine ships only the generic `flat` set:

Setting	Read by	Meaning
`OMDRC_SITE_DATA_DIRS`	CMake	semicolon-separated search path for `configs/<geo>` + `filters/<geo>`; first match wins
`OMDRC_SITE_ROOT`	the design scripts	the checkout they read and write room data in; also `--site-root`

Both default to the engine checkout, which is the historical single-repository layout. Everything in this document is unchanged by the split: the manifest records paths relative to the geometry root, so a bundle verifies identically wherever that root happens to be. What does change is *which* repository the deployment commit belongs to, and which working directory each handoff step names – the tooling prints both.

`verify_filter_bundle.py --site-root DIR` is what makes a manifest's recorded `configs/<geometry>/...` template path resolve in the checkout the set came from; CMake passes it automatically.

11 Installation requirement

`cmake/core-drc.cmake` installs each runtime manifest and analysis JSON beside the RAWs, while deliberately excluding `rew/`, `source/`, and source recipes. The full exports remain development-only because their exact hashes and parsed metadata are embedded in the installed manifest, and their numbers are already in the installed analysis file.

CMake runs the read-only verifier before copying a geometry. It also rejects every new `@design` config that lacks its same-named provenance manifest. A broken bundle therefore fails configuration/packaging rather than producing a UI that looks authoritative; legacy variants without manifests remain installed only for compatibility and are visibly unverified in the response page.